Lab2 Bomblab 实验报告

顾心怡

第 1 关　函数调用、参数传递、返回值

先分析汇编的整体框架，函数调用分支、循环、比较和返回的位置。

```
0000000000402020 <phase_01>:
  402020: endbr64
  402024: push   %rbp
  402025: mov    %rsp,%rbp
  402028: push   %r12
  40202a: mov    %rsi,%r12    //r12=y
  40202d: mov    $0x1,%esi
  402032: push   %rbx
  402033: mov    %rdi,%rbx    //rbx = 第一个参数 input 字符串指针
  402036: sub    $0x10,%rsp#建立栈帧
……
  402073: sete   %al              //相等 al=1
  402076: add    $0x10,%rsp       //恢复栈
  40207a: pop    %rbx
  40207b: pop    %r12
  40207d: pop    %rbp
  40207e: ret
```

phase_01 中共调用了三个函数

1、调用<valid_u32_fields>测试输入格式。如果格式错误，则直接 pop 恢复栈

```
  40203f: test   %al,%al  //调用 valid_u32_fields 测试输入格式。如果 al=0，则格式错误，进入分支一
  402041: jne    402050 <phase_01+0x30>    //jump not equal,如果不对，跳到“402050:  xor %eax,%eax”
  402043: add    $0x10,%rsp      //上面是语句判断，说明这里到 ret 中间是第一个分支？
  402047: xor    %eax,%eax    //分支一：不跳转顺序执行，直接 pop 恢复栈
  402049: pop    %rbx
  40204a: pop    %r12
  40204c: pop    %rbp
  40204d: ret
  40204e: xchg   %ax,%ax
  402050: xor    %eax,%eax//分支二：格式正确跳转
```

<__isoc99_sscanf@plt>

用于分析本关需要的整数个数。sscanf 调用后的返回值%eax 和 1 比较，如果 sscanf 返回值=1，继续。不等于 1，没读到合法输入。说明 phase_01 有一个输入。

```
40205e: call   4011e0 <__isoc99_sscanf@plt>
402063: cmp    $0x1,%eax
402066: jne    402043 <phase_01+0x23>
```

<p1_compute>中定义如何计算最后结果

```
402068: mov    %r12,%rdi                //第一个参数=challenge。从 r12 中取
40206b: call   401fe0 <p1_compute>
402070: cmp    %eax,-0x14(%rbp)         //比较正确的（p1_compute 的返回值）
```

vs 自己输入的整数

si *p1_compute

```
>0x401fe0 <p1_compute>          endbr64
 0x401fe4 <p1_compute+4>        push   %rbp
 0x401fe5 <p1_compute+5>        mov    %rdi,%rdx
 0x401fe8 <p1_compute+8>        mov    %rdi,%rax
 0x401feb <p1_compute+11>       shr    $0x20,%rax
 0x401fef <p1_compute+15>       mov    %rdx,%rsi
 0x401ff2 <p1_compute+18>       mov    %eax,%edi
 0x401ff4 <p1_compute+20>       shr    $0x10,%rsi
 0x401ff8 <p1_compute+24>       mov    %rsp,%rbp
 0x401ffb <p1_compute+27>       call   0x401fc0 <p1_step1>
 0x402000 <p1_compute+32>       mov    %edi,%esi
 0x402002 <p1_compute+34>       mov    %edx,%edi
 0x402004 <p1_compute+36>       mov    %eax,%ecx
 0x402006 <p1_compute+38>       call   0x401fb0 <p1_step0>
 0x40200b <p1_compute+43>       mov    %ecx,%esi
 0x40200d <p1_compute+45>       mov    %eax,%edi
 0x40200f <p1_compute+47>       call   0x401fd0 <p1_step2>
 0x402014 <p1_compute+52>       pop    %rbp
 0x402015 <p1_compute+53>       ret
 0x402016                       cs nopw 0x0(%rax,%rax,1)
```

先确定 p1_compute 里三次调用的实际参数，写出大框架：

从 64 位 challenge 切出三个 32 位字段

p1_step1(hi, mid), p1_step0(lo, hi), p1_step2(s0, s1)

```
lo  = (uint32_t)(challenge);          // 位 0..31
mid = (uint32_t)(challenge >> 16);    // 位 16..47
hi  = (uint32_t)(challenge >> 32);    // 位 32..63
```

继续分别进入<p1_step0>：

```
0x401fb0 <p1_step0>     endbr64
0x401fb4 <p1_step0+4>   lea    (%rdi,%rsi,4),%eax
0x401fb7 <p1_step0+7>   xor    $0x60e0,%eax
0x401fbc <p1_step0+12>  ret
0x401fbd                nopl   (%rax)
```

eax = rdi + rsi*4 = x + 4*y

eax ^= 0x60e0

则 p1_step0(x, y) = (x + 4*y) ^ 0x60e0

<p1_step1>：

```
0x401fc4 <p1_step1+4>   lea    (%rdi,%rdi,2),%eax
0x401fc7 <p1_step1+7>   add    $0x26fb,%esi
0x401fcd <p1_step1+13>  xor    %esi,%eax
0x401fcf <p1_step1+15>  ret
```

eax = rdi + rdi*2 = 3*x //32 位的 eax 会清零高 32 位，只保存低 32 位
esi = y + 0x26fb
eax = (3*x) ^ (y + 0x26fb)
则 p1_step1(x, y) = (3*x) ^ (y + 0x26fb)
<p1_step2>：

```
0x401fd4 <p1_step2+4>   lea    (%rdi,%rdi,2),%eax
```

```
0x401fd7 <p1_step2+7>   add    %esi,%eax
0x401fd9 <p1_step2+9>   xor    $0x9a8e,%eax
0x401fde <p1_step2+14>  ret
```

eax = 2*x+x=3*x
eax = 3*x + y
eax ^= 0x9a8e
p1_step2(x, y) = (3*x + y) ^ 0x9a8e

所以具体的运算是：

```
uint32_t s1 = (3u* hi) ^ (mid + 0x26fb);          // p1_step1(hi, mid)
uint32_t s0 = (lo + 4u * hi) ^ 0x60e0;            // p1_step0(lo, hi)
uint32_t answer = (3u * s0 + s1) ^ 0x9a8e;        // p1_step2(s0, s1)
```

在 gdb 上验证：

```
break *p1_step1; break *p1_step0; break *p1_step2
run --seed 42 --phase 1 < input.txt
```

在每个函数调用前停下，看返回值是否与推导的关系式一致
Phase 1 challenge: 0x28efe333b266f103

```
p/u $edi        # 应等于 hi
p/u $esi        # 应等于 mid
```

```
=> 0x401fc0 <p1_step1>: endbr64
(gdb) p/u $edi
$1 = 686809907
(gdb) p/u $esi
$2 = 3811816038
(gdb) finish
Run till exit from #0  0x0000000000401fc0 in p1_step1 ()
0x0000000000402000 in p1_compute ()
1: x/i $pc
=> 0x402000 <p1_compute+32>:    mov    %edi,%esi
(gdb) p/u $eax
$3 = 2583458040
```

p/u $edi //lo ; p/u $esi //hi; finish;　　p/u $eax　// s0

```
=> 0x401fb0 <p1_step0>: endbr64
(gdb) p/u $edi
$4 = 2993090819
(gdb) p/u $esi
$5 = 686809907
(gdb) finish
Run till exit from #0  0x0000000000401fb0 in p1_step0 ()
0x000000000040200b in p1_compute ()
1: x/i $pc
=> 0x40200b <p1_compute+43>:    mov    %ecx,%esi
(gdb) p/u $eax
$6 = 1445338415
```

p/u $edi　// s0; p/u $esi // s1 ;finish ;p/u $eax　　//最终答案

```
=> 0x401fd0 <p1_step2>: endbr64
(gdb) p/u $edi
$7 = 1445338415
(gdb) p/u $esi
$8 = 2583458040
(gdb) finish
Run till exit from #0  0x0000000000401fd0 in p1_step2 ()
0x0000000000402014 in p1_compute ()
1: x/i $pc
=> 0x402014 <p1_compute+52>:    pop    %rbp
(gdb) p/u $eax
$9 = 2624475659
```

输入不同的值测试：

```
sunny@sunny:~/ics-2026-bomblab$ ./defuser --seed 281470681743360 --phase 1 | ./bomb --seed 281470681743360 --phase 1

Phase 1 challenge: 0xd382fcb86194b6ae
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

```
sunny@sunny:~/ics-2026-bomblab$ python3 public_test.py --phase 1
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

第 2 关　　数组、循环、递推关系

分析本关需要的整数个数。此处比较 eax 是否=6，猜测本关有 6 个输入。

```
4020dc: xor    %eax,%eax
4020de: call   4011e0 <__isoc99_sscanf@plt>
4020e3: pop    %rdx
4020e4: pop    %rcx
4020e5: cmp    $0x6,%eax
```

继续分析下面汇编，用 6 次 lea 把（%rbp+偏移量）地址的数值存储到不同寄存器中，共 6 个，会使用栈保存。由于压栈顺序为从右到左，栈是向下增长的，所以用数组表示对应关系就是：

rbp-0x40 a[0]
rbp-0x3c a[1]
rbp-0x38 a[2]
rbp-0x34 a[3]
rbp-0x30 a[4]
rbp-0x2c a[5]

```
4020ba: lea     -0x2c(%rbp),%rax
4020be: lea     -0x3c(%rbp),%rcx
4020c2: mov     $0x403173,%esi //
//此处的0x403173, 大概率是%u %u %
4020c7: push    %rax
4020c8: lea     -0x30(%rbp),%rax
4020cc: lea     -0x40(%rbp),%rdx
4020d0: mov     %r12,%rdi
4020d3: push    %rax
4020d4: lea     -0x34(%rbp),%r9 /
4020d8: lea     -0x38(%rbp),%r8 /
```

整个过程是

```
4020dc: xor     %eax,%eax
4020de: call    4011e0 <__isoc99_sscanf@plt>
4020e3: pop     %rdx
4020e4: pop     %rcx
4020e5: cmp     $0x6,%eax
4020e8: jne     4020a5 <phase_02+0x25>
4020ea: mov     %r13,%rax
4020ed: mov     -0x40(%rbp),%esi
4020f0: shr     $0x20,%rax
4020f4: and     $0x7f,%eax
4020f7: xor     $0x8,%eax
4020fa: cmp     %eax,%esi
4020fc: jne     4020a5 <phase_02+0x25>
4020fe: mov     $0xa,%ecx
402103: mov     $0x1,%edx
402108: cmp     $0x6,%rdx
40210c: je      4020a7 <phase_02+0x27>
40210e: mov     %r13,%rax
402111: shr     %cl,%rax
402114: add     $0x5,%ecx
402117: and     $0x3f,%eax
40211a: xor     %edx,%eax
40211c: lea     (%rax,%rsi,2),%eax
```

40211f: mov -0x40(%rbp,%rdx,4),%esi
402123: add $0x1,%rdx
402127: cmp %eax,%esi
402129: je 402108 <phase_02+0x88>
40212b: jmp 4020a5 <phase_02+0x25>

1. 从 r13 取出原始 challenge；
2. 用 ecx 里的移位量，把 challenge 右移 shift 位；
3. 取低 6 位，再异或下标 i，得到本轮的常数 t；
4. 用 rsi 里存的「上一个值」× 2，加上 t，得到本轮的正确值，存在 eax；
5. 把输入的第 i 个值加载到 rsi（为下一轮递推做准备）；
6. 下标 i 加 1；
7. 比较：计算出来的正确值 == 输入值？相等就跳回循环条件，继续下一轮。即

```
for (i = 1; i <= 5; i++) {
    shift = 5*(i+1);                          // 10,15,20,25,30
    t = ((challenge >> shift) & 0x3F) ^ i;    // 取 6 位再异或下标
    values [i] = 2 * values [i - 1] + t       // 前一项 ×2 + 本轮常数
}
```

Seed=42 的时候，v0 = ((challenge >> 32) & 0x7F) ^ 0x08，是 95，和之前的分析匹配

```
(gdb) x/6uw $rbp - 0x40
0x7fffffffd2f0: 0       0       0       0
0x7fffffffd300: 0       0
(gdb)
```

```
=> 0x402080 <phase_02>: endbr64
(gdb) p/x $r13
$1 = 0x0
(gdb) x/s $rdi
0x7fffffffd3d0: "95 0 0 0 0 0"
```

通过 gdb 调试，得到当 seed=42 时，输出的 6 个数为：95 228 485 10212055 4135

```
sunny@sunny:~/ics-2026-bomblab$ gdb --args ./bomb --seed 42 --phase 2
第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

测试其他 seed：./defuser --seed 281470681743360 --phase 2 | ./bomb --seed

281470681743360 --phase 2

```
Phase 2 challenge: 0x38b7974814ba780f
Input (unsigned decimal integers):
Phase 2 defused.
```

```
sunny@sunny:~/ics-2026-bomblab$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

第 3 关 switch 分支与跳转表

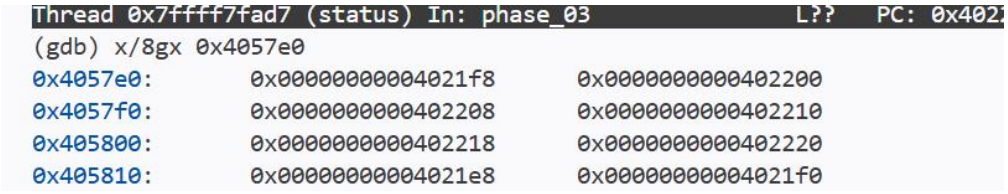
这关是关于 switch 分支和跳转表的。所以主要是解读跳转表的各个运算关系。首先通过 call <__isoc99_sscanf@plt>后的 “cmp $0x2,%eax”和读取 $0x40317f,%esi 处的内容得到 “%u %u”,确定是两个输入。

```
(gdb) x/s 0x40317f
0x40317f:       "%u %u"
```

```
0000000000402230 <phase_03>:
  402262: lea    -0x14(%rbp),%rcx
  402266: lea    -0x18(%rbp),%rdx
  40226a: mov    $0x40317f,%esi
  40226f: mov    %rbx,%rdi
  402272: call   4011e0 <__isoc99_sscanf@plt>
  402277: cmp    $0x2,%eax
  40227a: jne    402253 <phase_03+0x23>
  40227c: mov    %r12d,%edi
  40227f: xor    %eax,%eax
  402281: and    $0x7,%edi
  402284: cmp    %edi,-0x18(%rbp)
  402287: jne    402255 <phase_03+0x25>
  402289: mov    %r12,%rax
  40228c: mov    %r12,%rsi
  40228f: shr    $0x3,%rax
  402293: shr    $0x20,%rsi
  402297: xor    %eax,%esi
  402299: call   4021d0 <p3_dispatch>
  40229e: cmp    %eax,-0x14(%rbp)
  4022a1: sete   %al
  4022a4: jmp    402255 <phase_03+0x25>
```

为了确定具体 table[selector] 的分支跳转到哪个 case，要读取 table[selector]里存的地址。由 *4021dd:* notrack jmp *0x4057e0(,%rax,8)得到跳转表的首地址是

0x4057e0。用 x/8gx 读取到每行 2 个值，行首地址是左边那个的地址，右边那个在 +8。再对应到具体<p3_case_i>的地址，得到 selector 与 case 的对应关系是 0→case6, 1→case1, 2→case5, 3→case4, 4→case7, 5→case3, 6→case0, 7→case2



uint32_t selector = (uint32_t)challenge & UINT32_C(0x7);
uint32_t v = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);

结合 <p3_dispatch>看具体的运算关系:

```
  4021d4: mov     %edi,%eax
  4021d6: mov     %esi,%edi
  4021d8: cmp     $0x7,%eax
  4021db: ja      402225 <p3_dispatch+0x55>
  4021dd: notrack jmp *0x4057e0(,%rax,8)
  4021e5: nopl    (%rax)
  4021e8: jmp     402130 <p3_case_0>
  4021ed: nopl    (%rax)
  4021f0: jmp     402160 <p3_case_2>
  4021f5: nopl    (%rax)
  4021f8: jmp     4021a0 <p3_case_6>
  4021fa: nopw    0x0(%rax,%rax,1)
  402200: jmp     402150 <p3_case_1>
  402205: nopl    (%rax)
  402208: jmp     402190 <p3_case_5>
  40220a: nopw    0x0(%rax,%rax,1)
  402210: jmp     402180 <p3_case_4>
  402215: nopl    (%rax)
  402218: jmp     4021c0 <p3_case_7>
  40221a: nopw    0x0(%rax,%rax,1)
  402220: jmp     402170 <p3_case_3>
  402225: xor     %eax,%eax
 <p3_case_0>:
  402134: mov     %edi,%eax
  402136: shr     $0x2,%edi
  402139: xor     $0x6e16,%eax
  40213e: add     %edi,%eax
case 6: result = (v ^ UINT32_C(0x6e16)) + (v >> 2); break; /* p3_case_0 */
 <p3_case_1>:
  402154: lea     0xaa78(%rdi,%rdi,4),%eax
即 case 1: result = (5u * v) + UINT32_C(0xaa78); break; // p3_case_1
<p3_case_2>:
```

```
  402164: lea      0xc2fa(%rdi,%rdi,2),%eax
```
即 case 7: result = (3u * v) + UINT32_C(0xc2fa); break; /* p3_case_2 */
```
<p3_case_3>:
  402174: lea      (%rdi,%rdi,1),%eax
  402177: xor      $0x57eb,%eax
```
即 case 5: result = (2u * v) ^ UINT32_C(0x57eb); break; /* p3_case_3 */
```
<p3_case_4>:
  402184: lea      0x81b0(%rdi,%rdi,4),%eax
```
即 case 3: result = (5u * v) + UINT32_C(0x81b0); break; /* p3_case_4 */
```
 <p3_case_5>:
  402194: lea      0x4b7cb(%rdi,%rdi,4),%eax
```
即 case 2: result = (5u * v) + UINT32_C(0x4b7cb); break; /* p3_case_5 */
```
 <p3_case_6>:
  4021a4: lea      0x0(,%rdi,4),%eax
  4021ab: xor      $0xc13a,%eax
```
即 case 0: result = (4u * v) ^ UINT32_C(0xc13a); break; //p3_case_6
```
 <p3_case_7>:
  4021c4: mov      %edi,%eax
  4021c6: shr      $1,%edi
  4021c8: xor      $0x6a4c,%eax
  4021cd: add      %edi,%eax
```
即 case 4: result = (v ^ UINT32_C(0x6a4c)) + (v >> 1); break; /* p3_case_7 */

用 gdb 调试验证

break *phase_03 ； break *p3_dispatch ;gdb ./bomb

p/x $challenge// 得到 0x581ce1ff0e4ae394

break *0x402284

break *p3_dispatch

continue

p/u $edi # 先看 selector 是多少

set {unsigned int}($rbp-0x18) = $edi # 把栈上的第 1 个输入直接改成 selector

continue，得到 edi 为 4

```
Breakpoint 14, 0x0000000000402284 in phase_03 ()
1: x/i $pc
=> 0x402284 <phase_03+84>:      cmp    %edi,-0x18(%rbp)
(gdb) p/u $edi
$2 = 4
(gdb)
```

```
(gdb) x/4wx $rbp-0x18
0x7fffffffd338: 0x00000004      0x00000000      0x00000003      0x00000000
(gdb)
```

为了验证 selector=4 时会跳转到哪个 case，所以先在每个 case 位置打上断点，continue，发现停在了 case7 的位置，与推理一致

```
multi-thre Thread 0x7ffff7fad7 (asm) In: p3_case_7
Note: breakpoint 10 also set at pc 0x4021c0.
Breakpoint 24 at 0x4021c0
(gdb) continue
Continuing.

Breakpoint 10, 0x00000000004021c0 in p3_case_7 ()
1: x/i $pc
=> 0x4021c0 <p3_case_7>:        endbr64
(gdb)
```

Finish 后得到的返回值 p/u $eax

```
=> 0x40229e <phase_03+110>:
(gdb) p/u $eax
$5 = 381728391
```

然后尝试其他 seed，看别的分支的关系是否正确

```
sunny@sunny:~/ics-2026-bomblab$ python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

第 4 关　递归调用与返回过程

首先由 mov $0x1,%esi 再 call valid_u32_fields 和 cmp $0x1,%eax，说明输入是 1 个无符号整数。

phase_04 中包含的函数 p4_walk，所以先确定传入函数的参数分别是什么

r12 = challenge（进来时第 2 个参数，被存进 callee-saved 的 r12 保留）

由

402368: mov　%r12,%rcx//r12 中存的是 challenge

40236b: mov　%r12,%rax

40236e: mov　%r12d,%edi//edi=x>>32

402371: xor　%esi,%esi /%esi=0

402373: shr　$0x20,%rax//rax=challenge>>32（rax 存的是 r12）

402377: shr　$0x7,%rcx//rcx=challenge>>7

40237b: and　$0x7f,%edi//edi = challenge & 0x7f

40237e: mov　$0x7f,%edx//edx=0x7f=127

402383: xor　%eax,%ecx //ecx = (challenge>>7) ^ (challenge>>32)

402385: call　4022b0 <p4_walk>

得 x1=edi=challenge & 0x7f

x2=esi=0

x3=edx=0x7f=127

x4=ecx = (challenge>>7) ^ (challenge>>32)

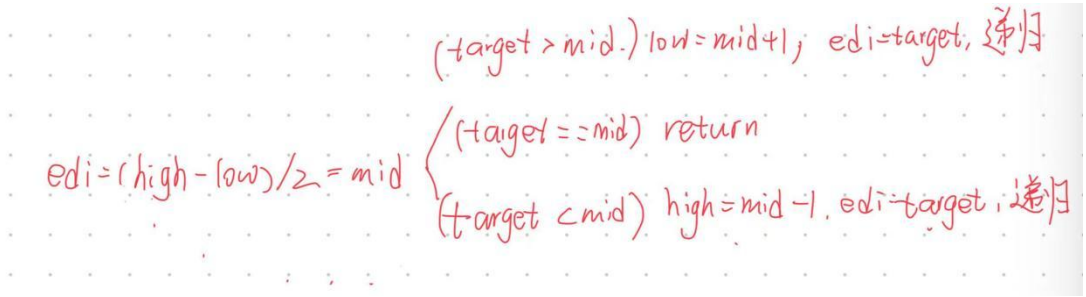
再分析每个参数的具体含义。由下面这段可知显示构造了一个中间值 mid，后续中间值不断和 eax 返回值做比较，可以推断是二分搜索法，eax 对应 target(要找的值)，下界 low，上界 high，x4

```
00000000004022b0 <p4_walk>:
  4022b4: mov     %edi,%eax//%eax=x1
  4022b6: mov     %edx,%edi//%edi=x3
  4022b8: sub     %esi,%edi//edi=x3-x2
  4022ba: shr     $1,%edi //edi = (x3-x2)/2
  4022bc: add     %esi,%edi//edi = x2+(x3-x2)/2，相当于(x2+x3)/2,= mid
  4022be: cmp     %eax,%edi //mid 和 x1 比较，若相等，则结束循环
  4022c0: je      402310 <p4_walk+0x60>
```

然后去找不同分支的判断条件和具体的运算

```
  4022c7: lea     (%rdi,%rcx,1),%ebx   //ebx=%rdi+%rcx=mid+x4
  4022ca: sub     $0x8,%rsp//%rsp-8
  4022ce: cmp     %edi,%eax //x1-mid
//如果 x1<mid,则走分支二。否则 x1>mid,esi=mid+1.可分析出 x1 相当于查找的 target
  4022d0: jb      4022f0 <p4_walk+0x40>
//target>mid  分支一：edx=mid+1
  4022d2: lea     0x1(%rdi),%esi          //分支一: esi = mid+1, 更新了搜索的范围
  4022d5: mov     %eax,%edi               //edi = target
  4022d7: call    4022b0 <p4_walk>        // p4_walk(target, mid+1, x3, x4)
  4022dc: lea     (%rax,%rax,2),%eax      //%eax=3%rax(返回值)
  4022df: xor     %ebx,%eax               //eax^=(mid+x4)
  4022e1: mov     -0x8(%rbp),%rbx
  4022e5: leave
  4022e6: xor     $0x19,%eax //eax^19
  4022e9: ret   //result = (3*sub)^(mid+x4) ^ 0x19
//target<mid  分支二：edx=mid-1
  4022f0: lea     -0x1(%rdi),%edx
  4022f3: mov     %eax,%edi //%edi=target
  4022f5: call    4022b0 <p4_walk> //p4_walk(target, x2, mid-1, x4)
  4022fa: lea     (%rax,%rax,2),%eax ////%eax=3%rax
  4022fd: xor     %ebx,%eax //eax ^= (mid+x4)
  4022ff: mov     -0x8(%rbp),%rbx
  402303: leave
  402304: xor     $0x2,%eax //eax^=2
```

所以调用的不同情况如下：



子结果 → 3*子结果 → 异或(mid+x4) → 再异或(0x02/0x19)

参数	寄存器	
target	edi /x1	要索的目标值（0~127）
low	esi /x2	区间下界，初始 0
high	edx /x3	区间上界，初始 127
key	ecx /x4	每个节点要异或的变量

Gdb 调试
gdb --args ./bomb --seed 42 --phase 4
run --seed 42 --phase 4 < input4.txt
先确认 4 个参数（在 call 之前停下）break *0x402385

```
Breakpoint 5, 0x0000000000402385 in phase_04 ()
1: x/i $pc
=> 0x402385 <phase_04+101>:     call   0x4022b0 <p4_walk>
(gdb) p/u $edi
$1 = 114
(gdb) p/u $esi
$2 = 0
(gdb) p/u $edx
$3 = 127
(gdb) p/u $ecx
$4 = 3186935837
```

其中 edi =target 应等于 challenge & 0x7f

break *p4_walk 在递归调用函数处打断点，则每次调用都会停下，可以看到变量的变化。info registers 得到每层参数，再进行相关运算。经过 7 次递归后，找到了 target 值。

每层调用后参数的值的变化如下：
第 1 层
rcx 0xbdf4c81d 3186935837 //每次要异或的 x4
rdx 0x7f 127 //high，搜索区间的上界
rsi 0x0 0 //low，下界
rdi 0x72 114 //target，需要检索的值
此时 mid=(high+low)/2=63<target,进入分支一，更新 esi = mid+1=64
第 2、3、4、5、6、7 层

```
rcx            0xbdf4c81d          3186935837
rdx            0x7f                127
rsi            0x40                64
rdi            0x72                114
```

```
rcx            0xbdf4c81d          3186935837
rdx            0x7f                127
rsi            0x60                96
rdi            0x72                114
```

```
rcx            0xbdf4c81d          3186935837
rdx            0x7f                127
rsi            0x70                112
rdi            0x72                114
```

```
rcx            0xbdf4c81d          3186935837
rdx            0x76                118
rsi            0x70                112
rdi            0x72                114
```

```
rcx            0xbdf4c81d          3186935837
rdx            0x72                114
rsi            0x70                112
rdi            0x72                114
```

```
rcx            0xbdf4c81d          3186935837
rdx            0x72                114
rsi            0x72                114
rdi            0x72                114
```

刚开始以为 target=114 是答案，但输入到 input4 中发现失败了。分析 *40238a:* cmp %eax,-0x14(%rbp) 可知输入的 1 个数要等于 p4_walk 的返回值（存放在%eax）。

Break *0x40238a 并读取$eax 的值得知 1434892390。验证后发现确实是 seed 42 的答案。

用不同 seed 验证

```
./defuser --seed 4756332  --phase 4 | ./bomb --seed 4756332 --phase 4
Phase 4 challenge: 0x9e48e687/9625910
Input (unsigned decimal integers):
Phase 4 defused.
```

```
  for s in 0 1 42 63 64 100 127 987654321; do …
  Phase 4 defused.
```

```
sunny@sunny:~/ics-2026-bomblab$ python3 public_test.py --phase 4
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
```

第 5 关　结构体布局、指针、链表遍历

由 cmp $0x2,%eax 得 sscanf 写入两个地址，由具体代码得第一个输入是 rdx=rbp-0x1c、第二个输入是 rcx=rbp-0x18。

```
 402432: lea     -0x18(%rbp),%rcx //rcx=(rbp-18)中的值
  402436: lea     -0x1c(%rbp),%rdx //rdx=(rbp-1c)中的值
  40243a: mov     $0x40317f,%esi
  40243f: mov     %rbx,%rdi
  402442: call    4011e0 <__isoc99_sscanf@plt>
  402447: cmp     $0x2,%eax
```

调用 p5_traverse 时，

```
40244c: lea     -0x14(%rbp),%rsi   //rsi=y=-0x14(%rbp)，让函数写结果的地址？
402450: mov     %r12,%rdi          //rdi=x=challenge
402453: call    4023a0 <p5_traverse>
```

调用后做两次比较

第一次：

```
402458: mov     %eax,%edx //先将 p5_traverse 的返回值 eax 存到%edx 中
40245a: xor     %eax,%eax
40245c: cmp     %edx,-0x1c(%rbp)//可知 rbp-0x1c 必须等于 edx。即第一个输入必须等于 p5_traverse 的返回值。
```

第二次比较：

```
  402461: mov     -0x14(%rbp),%eax
  402464: cmp     %eax,-0x18(%rbp)//第 2 个输入必须等于函数写到（rbp-14）的值
  402467: sete    %al
  40246a: jmp     402425 <phase_05+0x25>
```

已经在实验文档中知道本关是关于结构体布局、指针和链表遍历，所以在 p5_traverse 具体找指针、结构体和遍历终止条件。先分析需要用到的变量具体如何由 challenge 得出：

```
①mov     %rdi,%rax      //rax=challenge
    mov     %rdi,%rcx
  mov     %rsi,%r8       //r8=y=-0x14(%rbp)
  and     $0x7,%eax      // eax&7
```

令 idx = challenge & 7，功能是作为下标对应具体选哪个链表

```
②由 mov     %rdi,%rax    ; shr   $0x20,%rdi; shr     $0x6,%rax
后 xor %edi,%eax  得返回值 eax= (challenge>>6)^(challenge>>32)，记作 mix
③ shr     $0x3,%rcx    ；mov     %ecx,%edi
and     $0x3,%edi    得 edi=(challenge>>3)&3，记作 count
```

再分析链表顺序：

```
4023b4: mov     0x405b20(,%rax,8),%rdx   //rdx=table[challenge&7]
```

结合 gdb 调试读取表首地址为 0x405b20 的指针表：

```
(gdb) x/8gx 0x405b20
0x405b20 <p5_heads>:    0x0000000000405bb0      0x0000000000405c00
0x405b30 <p5_heads+16>: 0x0000000000405b60      0x0000000000405b90
0x405b40 <p5_heads+32>: 0x0000000000405ba0      0x0000000000405c10
0x405b50 <p5_heads+48>: 0x0000000000405bf0      0x0000000000405be0
```

可知是一个有 8 个节点的链表，每个节点指向一个结构体。

```
(gdb) x/4wx 0x0000000000405bb0
0x405bb0 <p5_nodes+80>: 0x0000031e      0x00000582      0x00405c00      0x00000
000
(gdb) x/gx 0x0000000000405bb0 + 8
0x405bb8 <p5_nodes+88>: 0x0000000000405c00
```

```
(gdb)  x/3wx 0x00405c00
0x405c00 <p5_nodes+160>:        0x0000a307      0x00000443      0x00405b60
(gdb) x/3wx 0x00405b60
0x405b60 <p5_nodes>:    0x00009dab      0x0000081d      0x00405b90
```

链表用 C 语言定义就是：

```
static const struct p5n p5n_nodes[12] = {
	{ UINT32_C(0x031e), UINT32_C(0x0582), &p5n_nodes[1]  }, /* N5  =
heads[0] */
	{ UINT32_C(0xa307), UINT32_C(0x0443), &p5n_nodes[2]  }, /* N10 =
heads[1] */
	{ UINT32_C(0x9dab), UINT32_C(0x081d), &p5n_nodes[3]  }, /* N0  =
heads[2] */
	{ UINT32_C(0x1bda), UINT32_C(0x0717), &p5n_nodes[4]  }, /* N3  =
heads[3] */
	{ UINT32_C(0x4f9d), UINT32_C(0x036f), &p5n_nodes[5]  }, /* N4  =
heads[4] */
	{ UINT32_C(0x20cb), UINT32_C(0x091f), &p5n_nodes[6]  }, /* N11 =
heads[5] */
	{ UINT32_C(0xcea9), UINT32_C(0x0a52), &p5n_nodes[7]  }, /* N9  =
heads[6] */
	{ UINT32_C(0x7b26), UINT32_C(0x017e), &p5n_nodes[8]  }, /* N8  =
heads[7] */
	{ UINT32_C(0xbff2), UINT32_C(0x0656), &p5n_nodes[9]  }, /* N2 */
	{ UINT32_C(0xaa69), UINT32_C(0x0b39), &p5n_nodes[10] }, /* N1 */
	{ UINT32_C(0xbc93), UINT32_C(0x000a), &p5n_nodes[11] }, /* N7 */
	{ UINT32_C(0x022b), UINT32_C(0x0205), NULL                  }  /* N6  链
尾 */
};
static const struct p5n *const p5n_heads[8] = {
	&p5n_nodes[0], &p5n_nodes[1], &p5n_nodes[2], &p5n_nodes[3],
	&p5n_nodes[4], &p5n_nodes[5], &p5n_nodes[6], &p5n_nodes[7]
};
```

进一步分析结构体，需要知道每个节点指针 rdx 对应多少的个结构成员。代码中有 (%rdx), 0x4(%rdx), 0x8(%rdx)，所以对应有三个结构体成员。令(%rdx)对应 p->a,0x4(%rdx)对应 p->b, 0x8(%rdx)对应 p->c。分析三个成员具体的功能：

```
4023c7: mov    0x4(%rdx),%esi   //esi=head.b, 第二个结构体成员
4023cc: xor    (%rdx),%eax    //eax=mix^rdx
4023d0: xor    $0xa205,%eax    //eax ^= 0xa205
4023d5: mov    %esi,(%r8)      //*out=head.b
4023d8: add    %esi,%eax       //eax = (mix ^ head.a ^ 0xa205) + head.b
4023dd: je     4023f9 <p5_traverse+0x59> //如果 count==0, 返回。
```

如果不相等，继续执行。更新下标和相关变量，准备进行新一轮的查找，实现链表遍历：

```
4023df: mov    $0x1,%ecx
4023e4: mov    0x8(%rdx),%rdx     //p=p->c
4023e8: add    $0x1,%ecx          //i ++
4023eb: mov    0x4(%rdx),%esi     //esi = p->b
4023ee: xor    (%rdx),%eax        //eax^=p->a
4023f0: add    %esi,%eax          // eax^+=p->b
4023f2: mov    %esi,(%r8)         //r8=p->b
4023f5: cmp    %ecx,%edi          //count=count-i
4023f7: jae    4023e4 <p5_traverse+0x44>
4023f9: ret
```

由于每次 mov 0x8(%rdx),%rdx 后 ecx 都会加一，可猜测 0x8(%rdx）类似于 C 语言中的 next 指针，指向后继节点，C 语言结构体是：

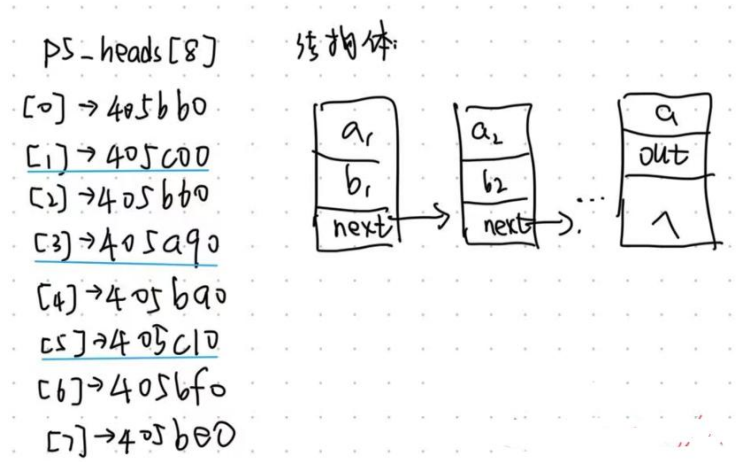
```
struct p5n {
    uint32_t a;
    uint32_t b;
    const struct p5n *next;
};
```

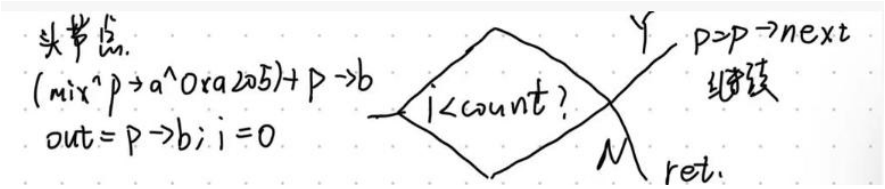
变量定义和链表遍历：

```
static uint32_t p5_traverse(uint64_t challenge, uint32_t *out) {
    uint32_t idx   = (uint32_t)challenge & UINT32_C(0x7);
    uint32_t count = (uint32_t)(challenge >> 3) & UINT32_C(0x3);
    uint32_t mix   = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
    const struct p5n *p = p5n_heads[idx];
    uint32_t acc = (mix ^ p->a ^ UINT32_C(0xa205)) + p->b;
    *out = p->b;
    for (uint32_t i = 0; i < count; ++i) {
        p = p->next;
        acc ^= p->a;
        acc += p->b;
        *out = p->b;
    }
    return acc;
```

}

链表结构如下：






```
int solve_phase_5(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;
    uint32_t last_b;
    uint32_t acc = p5_traverse(challenge, &last_b);
    values[0] = acc;            /* 第 1 个数：返回值 */
    values[1] = last_b;         /* 第 2 个数：最后一个节点的 b */
    return write_answer(out, cap, values, count);
}
```

Gdb 验证

```
sunny@sunny:~/ics-2026-bomblab$ ./defuser --seed 888888889 --phase 5 | ./bomb --seed 888888889 --phase 5

Phase 5 challenge: 0x2f1f9b4a5cc64925
Input (unsigned decimal integers):
Phase 5 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 已解除] [6 待解除]
```

```
sunny@sunny:~/ics-2026-bomblab$ python3 public_test.py --phase 5
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
```

第 6 关　函数指针表与间接调用

```
0000000000402550 <phase_06>:
  402554: push   %rbp
  402555: mov    %rsp,%rbp
  402558: push   %r12
  40255a: mov    %rsi,%r12
  40255d: mov    $0x1,%esi
  402562: push   %rbx
  402563: mov    %rdi,%rbx
  402566: sub    $0x10,%rsp
  40256a: call   402c00 <valid_u32_fields>
  40256f: test   %al,%al
  402571: jne    402580 <phase_06+0x30>
  402573: add    $0x10,%rsp
  402577: xor    %eax,%eax
  402579: pop    %rbx
  40257a: pop    %r12
  40257c: pop    %rbp
  40257d: ret
  40257e: xchg   %ax,%ax
  402580: xor    %eax,%eax
  402582: lea    -0x14(%rbp),%rdx
  402586: mov    $0x403182,%esi
  40258b: mov    %rbx,%rdi
  40258e: call   4011e0 <__isoc99_sscanf@plt>
  402593: cmp    $0x1,%eax
  402596: jne    402573 <phase_06+0x23>
  402598: mov    %r12,%rdi
  40259b: call   402500 <p6_dispatch>
  4025a0: cmp    %eax,-0x14(%rbp)
  4025a3: sete   %al
  4025a6: add    $0x10,%rsp
  4025aa: pop    %rbx
  4025ab: pop    %r12
  4025ad: pop    %rbp
```

本关卡有一个输入：

```
(gdb) x/s 0x403182
0x403182:       "%u"
```

具体分析函数<p6_dispatch>的指针表与间接调用：

402509: mov %rdi,%rbx /=challenge

40250c: shr $0x6,%rdi /rdi=challenge>>6

402510: mov %rbx,%rax /rax=rbx

402513: shr $0x20,%rax /rax=challenge>>32，取高 32 位

402517: xor %eax,%edi /32 位异或： eax 返回值^edi,即(x>>32)^(x>>6)的低 32 位=arg1，作为第一次调用的参

402519: mov %rbx,%rax /rax=x，重新初始化 rax

40251c: sub $0x8,%rsp /栈对齐

402520: and $0x38,%ebx /0x38&%ebx 即 ebx=x&0x38（0b00111000，即 x 的 bits 5..3 左移 3 位）

402523: and $0x7,%eax /eax=x&7=idx1,第一次查表索引

402526: call *0x405dc0(,%rax,8) /①第一次间接调用,函数=table[idx1]。从 0x405dc0 + rax×8 取函数指针并调用，rdi=arg1，返回值在 eax

40252d: xor $0x118f,%eax //②eax = eax ^ 0x118f

402532: mov %eax,%edi /把异或结果作为第二次调用的参数

402534: call *0x405dc0(%rbx)//③ 第二次间接调用，函数 = table[idx2]。从 0x405dc0 + rbx 取函数指针

40253a: mov -0x8(%rbp),%rbx

40253e: leave

40253f: add $0xda01,%eax

402544: ret //eax=输入的数

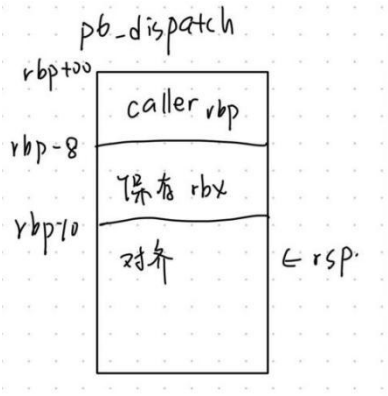
通过 64 位 seed x 的低 6 位在 0x405dc0 函数指针表中选择两个函数（idx1=x [2:0] 和 idx2=x [5:3]），以(x>>6)^(x>>32)为第一个函数的输入，经第一次调用→异或 0x118f →第二次调用 →加 0xda01 后得到目标值（等于输入的无符号整就 defused）。第一次间接调用的地址是*0x405dc0(,%rax,8)，第二次是 *0x405dc0(%rbx)

还原成 C 语言是：

```
//p6_dispatch
static uint32_t p6_dispatch(uint64_t x)
{
    uint32_t arg1 = (uint32_t)(x >> 6) ^ (uint32_t)(x >> 32);
    uint32_t idx1 = (uint32_t)(x & 0x7u);
    uint32_t idx2 = (uint32_t)((x >> 3) & 0x7u);

    uint32_t a = p6_ops[idx1](arg1);  /* ① 第一次间接调用 */
    a ^= 0x118fu;                     /* ② 异或密钥 */
    uint32_t b = p6_ops[idx2](a);     /* ③ 第二次间接调用 */
```

```
    return b + 0xda01u;                 /* ④ 加偏移 */
}
```



具体看 8 个函数指针：

```
(gdb) x/8gx 0x405dc0
0x405dc0 <p6_ops>:      0x00000000004024e0      0x00000000004024d0
0x405dd0 <p6_ops+16>:   0x0000000000402470      0x00000000004024a0
0x405de0 <p6_ops+32>:   0x0000000000402490      0x00000000004024a0
0x405df0 <p6_ops+48>:   0x00000000004024c0      0x00000000004024e0
```

根据函数指针，反汇编出具体的汇编语言，得到具体的运算关系并分析。

```
<p6_op_0+0>(table[2])
   0x0000000000402474 <p6_op_0+4>:      lea     (%rdi,%rdi,1),%edx
/edx=2edi=2v
   0x0000000000402477 <p6_op_0+7>:      lea     0x1267(%rdx),%eax
/eax=0x1267+2v
   0x000000000040247d <p6_op_0+13>:      add     %edi,%edx /edx=edx+edi=3v
   0x000000000040247f <p6_op_0+15>:     xor     %edx,%eax /eax^=eax
   0x0000000000402481 <p6_op_0+17>:      ret
   0x0000000000402482:   data16 cs nopw 0x0(%rax,%rax,1)
   0x000000000040248d:   nopl    (%rax)
  return (2*v + 0x1267) ^ (3*v);

   p6_op_1(table[4])
   0x0000000000402494 <p6_op_1+4>:      mov     %edi,%eax//eax=v
   0x0000000000402496 <p6_op_1+6>:      rol     $0x3,%eax
   0x0000000000402499 <p6_op_1+9>:      xor     $0xa720,%eax
   0x000000000040249e <p6_op_1+14>:      ret
   0x000000000040249f:   nop
return rol32(v,3) ^ 0xa720;
rol32(v,3) = (v<<3)|(v>>29)

p6_op_2 (table[3]、table[5])
   0x00000000004024a4 <p6_op_2+4>:      mov     %edi,%eax
```

```
   0x00000000004024a6 <p6_op_2+6>:        shr     $0x7,%eax//eax=v>>7
   0x00000000004024a9 <p6_op_2+9>:        xor     %edi,%eax
   0x00000000004024ab <p6_op_2+11>:       add     $0x200b,%eax
   0x00000000004024b0 <p6_op_2+16>:       ret
   0x00000000004024b1:   data16 cs nopw 0x0(%rax,%rax,1)
   0x00000000004024bc:   nopl     0x0(%rax)
return (v^(v >> 7)) +0x200b;
p6_op_3(table[6])
   0x00000000004024c4 <p6_op_3+4>:        lea     0x6753(%rdi),%eax
/eax=v+0x6753
   0x00000000004024ca <p6_op_3+10>:       shr     $0x3,%edi /edi=v>>3
   0x00000000004024cd <p6_op_3+13>:       xor     %edi,%eax
   0x00000000004024cf <p6_op_3+15>:       ret
   return (v + 0x6753) ^ (v >> 3);

p6_op_4 (table[1])
   0x00000000004024d4 <p6_op_4+4>:        mov     %edi,%eax /eax=v
   0x00000000004024d6 <p6_op_4+6>:        ror     $0x7,%eax
   0x00000000004024d9 <p6_op_4+9>:        ret
   0x00000000004024da:   nopw     0x0(%rax,%rax,1)
return ror32(v,7);
ror32(v,7) = (v>>7)|(v<<25)

p6_op_5 (table[0]、table[7])
   0x00000000004024e4 <p6_op_5+4>:        mov     %edi,%eax /eax=v
   0x00000000004024e6 <p6_op_5+6>:        shl     $0x7,%edi /edi=v<<7
   0x00000000004024e9 <p6_op_5+9>:        xor     $0xa1ed,%eax
   0x00000000004024ee <p6_op_5+14>:       add     %edi,%eax
   0x00000000004024f0 <p6_op_5+16>:       ret
   0x00000000004024f1:   data16 cs nopw 0x0(%rax,%rax,1)
   0x00000000004024fc:   nopl     0x0(%rax)
return (v ^ 0xa1ed) + (v<< 7);

0x405dc0 函数指针表里的顺序和 op 编号顺序不一致，对应下来是：
typedef uint32_t (*op_fn)(uint32_t);
static const op_fn p6_ops[8] = {
    p6_op_5,   /* table[0] 0x405dc0 → 0x4024e0 */
    p6_op_4,   /* table[1] 0x405dc8 → 0x4024d0 */
    p6_op_0,   /* table[2] 0x405dd0 → 0x402470 */
    p6_op_2,   /* table[3] 0x405dd8 → 0x4024a0 */
    p6_op_1,   /* table[4] 0x405de0 → 0x402490 */
    p6_op_2,   /* table[5] 0x405de8 → 0x4024a0 */
    p6_op_3,   /* table[6] 0x405df0 → 0x4024c0 */
    p6_op_5,   /* table[7] 0x405df8 → 0x4024e0 */
```

};
在两次 call 前打断点，看 seed=42 时的运行情况。
第一次 call 之前，rax = idx1，rdi = arg1
4024a0 是实际将跳转到的函数地址

```
Breakpoint 1, 0x0000000000402526 in p6_dispatch ()
1: x/i $pc
=> 0x402526 <p6_dispatch+38>:   call   *0x405dc0(,%rax,8)
(gdb) i r rax rdi
rax            0x5                 5
rdi            0x46fdbaa9          1191033513
(gdb) x/gx 0x405dc0+$rax*8
0x405de8 <p6_ops+40>:   0x00000000004024a0
```

第二次 call 之前
rbx = x&0x38 字节偏移，rdi = 异或后的参数
4024a0 是第二个函数的地址

```
=> 0x402534 <p6_dispatch+52>:   call   *0x405dc0(%rbx)
(gdb) i r rbx rdi
rbx            0x18                24
rdi            0x46707068          1181773928
(gdb) x/gx 0x405dc0+$rbx
0x405dd8 <p6_ops+24>:   0x00000000004024a0
```

```
sunny@sunny:~/ics-2026-bomblab$ ./defuser --seed 473
9562938 --phase 6 | ./bomb --seed 4739562938 --phase
 6
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0xdce45f79cc7094af
Input (unsigned decimal integers):
Phase 6 defused.
```

```
sunny@sunny:~/ics-2026-bomblab$ ./defuser --seed 4739562938 --phase 6 | .
/bomb --seed 4739562938 --phase 6

sunny@sunny:~/ics-2026-bomblab$ python3 public_test.py --phase 6
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
```

测试六关：

```
sunny@sunny:~/ics-2026-bomblab$ make test
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
```

Honor

先参考 teaching/07_stack_copy.c 如何实现同一个结构体内的相邻字段覆盖，观察局部对象的布局。Secret 关卡也和栈上相邻字段覆盖有关。

```
struct Frame {char scratch[16]; uint32_t marker; char answer[24]; };
```

结构体在 memory 里是紧挨着的，即 16 字节的 scratch[16]后就是 marker 的第一个字节。Marker 一共 4 字节，answer24 字节。

```
__attribute__((noinline)) uint32_t copy_demo(const char *source, size_t length) {
    struct Frame frame = {0};
    for (size_t index = 0; index < length; ++index)
        ((char *)&frame.scratch[0])[index] = source[index];
    return frame.marker;
}
int main(void) {
    static const char payload[] = "AAAAAAAAAAAAAAAADEMO";
    printf("scratch=%zu marker=%zu answer=%zu marker_value=%" PRIu32 "\n",
           offsetof(struct Frame, scratch), offsetof(struct Frame, marker),
           offsetof(struct Frame, answer), copy_demo(payload, sizeof(payload) - 1));
```

payload 是 6 个 A +DEMO+ 结尾\0，sizeof(payload) - 1，即只复制 20 个字节，不含\0

先根据 secret_gate、secret_copy 和 secret_phase 的汇编完整的。

Secret 的输入是一行不含换行的 ASCII 文本。先从 secret_gate、secret_copy 和 secret_phase 开始分析，构造第七行。

./defuser --seed 42 > secret.txt

输出为：

2624475659

95 228 485 1021 2055 4135

4 381728391

1434892390

917923980 2642

1191021204

echo 'test' >> secret.txtgdb ./bomb

(gdb) break *secret_gate(gdb) run --seed 42 < secret.txt

Honor 的 challenge 不会打印，进入 secret_gate，按调用约定，rdi 是 arg1 = 输入的第七行字符串（→rbx）。
Rsi（→r13，arg2）、rdx（→r14，arg3）是两个 64 位十六进制数，对应隐藏的 challenge 和 context
先 break *secret_gate，让程序停在初始的位置，然后查看参数寄存器

```
(gdb) x/s $rdi
0x7fffffffd520: "test"
(gdb) p/x $rsi
$1 = 0xccf635ee9e9e2fa4
(gdb) p/x $rdx
$2 = 0xdd4725171880807b
(gdb)
```

为了确认两个值，在 defuser 中临时调试 fprintf(stderr, "[dbg] challenge=%016llx context=%016llx\n", (unsigned long long)challenge, (unsigned long long)context);

```
sunny@sunny:~/ics-2026-bomblab$ ./defuser --seed 42 --secret 1>/dev/null
[dbg] challenge=ccf635ee9e9e2fa4 context=dd4725171880807b
```

gdb 验证得 rsi=challenge、$rdx==context
即 uint64_t A = challenge; uint64_t B = context;

0x4026b0 <secret_gate>:
mov $0x14,%ecx / ecx = 20
lea -0xd0(%rbp),%r12 / r12 = rbp-0xd0 = 缓冲区起点
mov %r12,%rdi
rep stos %rax,(%rdi) /rax=0：把 20×8=160 字节(0xa0)清零
movl $0x0,(%rdi) /末尾再补一个 0
mov %rbx,%rdi ; call strlen
sub $0x24,%rax /rax = strlen(行) - 36
cmp $0x7f,%rax ; jbe ... /判断长度，(strlen-36)≤127
mov %rbx,%rsi ; mov %r12,%rdi ; call secret_copy /把行复制到 rbp-0xd0
cmpl $0x4e45504f,-0xb0(%rbp) /rbp-0xb0 处必须等于 0x4e45504f
cmpl 是按小端读 4 字节，即把 0x4e45504f 这个数放进内存时，最低字节先放。所以内存里从地址低到高，实际排列是 4f 50 45 4e。根据 ASCII 的对应关系，得到十进制是 79 80 69 78 对应的 ASCII 字符是 OPEN。
mov %r14,%rdx ; mov %r13,%rsi ; lea -0xac(%rbp),%rdi
call secret_phase /用 rbp-0xac 处的字符串+ arg2/arg3

rbp-0xd0 是字符串复制的地址，而 rbp-0xb0 必须等于规定的 0x4e45504f，两者相差 32 字节，初步分析前 32 字节是输入的覆盖字符串。这与 07_stack_copy.c 类似。scratch[32] 之后跟一个 4 字节 marker，再之后是 answer。复制时只要让输入在偏移 32 处放上 OPEN，就覆盖了 marker 字段。



0x402690 <secret_copy>: 功能是复制字符串
movzbl (%rsi,%rax,1),%edx； /取 source[i]
mov %dl,(%rdi,%rax,1) ; /写到 dest[i]
add $0x1,%rax ; test %dl,%dl ; jne ... ; /直到遇到 0（连 \0 一起复制）

0x4025d0 <secret_phase>: endbr64
0x402610 : xor %eax,%eax
0x402612 : lea -0x24(%rbp),%r8
0x402616 : lea -0x28(%rbp),%rcx
0x40261a : mov $0x40317c,%esi
0x40261f : lea -0x2c(%rbp),%rdx
0x402623 : mov %rbx,%rdi
call 0x4011e0 <__isoc99_sscanf@plt>后 cmp $0x3,%eax 说明要输入 3 个%u，分别是：
rdx=rbp-0x2c→ s0 ; (lea -0x2c(%rbp),%rdx)
rcx=rbp-0x28 → s1 ; (lea -0x28(%rbp),%rcx)
r8 =rbp-0x24→ s2 ; (lea -0x24(%rbp),%r8)

在读入 3 个数后，要做 3 次比较。记 A = arg2（r12）、B = arg3（r13）
mov %r13d,%ecx ; and $0x3,%ecx ; cmp %ecx,-0x2c(%rbp) ; s0 要等于 B&3
imul $0xe395,%ecx,%r9d ; r9 = c*0xe395
mov %r12,%rsi 使得 rsi=A， shr $0x20,%rsi ; xor %r12d,%esi，即 rsi=(A>>32)^A，令 foldA=(A>>32)^A
mov %r13,%r8 ; shr $0x20,%r8 ; xor %r13d,%r8 ; foldB=(B>>32)^B
shr $0x10,%r13 ; xor %r13d,%r9d ; r9 ^= B>>16
mov %esi,%edi ; xor %r8d,%edi ; x = foldA^foldB
call secret_mix ; m = secret_mix(x)
xor %r9d,%eax ; cmp %eax,-0x28(%rbp) ; s1 需要等于 m^r9
shr $0x3,%esi ; foldA>>3
lea 0x3d11(%r8,%rcx,1),%edx ; add %esi,%edx ;
term=foldB+c+0x3d11+foldA>>3
xor %edx,%eax ; cmp -0x24(%rbp),%eax ; 检查 3: s2 =s1^term

```
uint64_t A = challenge;
uint64_t B = context;
uint32_t c       = (uint32_t)B & UINT32_C(0x3);
uint32_t foldA = (uint32_t)(A >> 32) ^ (uint32_t)A;
uint32_t foldB = (uint32_t)(B >> 32) ^ (uint32_t)B;
uint32_t x       = foldA ^ foldB;
uint32_t r9      = (c * UINT32_C(0xe395)) ^ (uint32_t)(B >> 16);
uint32_t m       = secret_mix_f(x);
uint32_t s1      = m ^ r9;
uint32_t term   = foldB + c + UINT32_C(0x3d11) + (foldA >> 3);
```

```
    uint32_t s2      = s1 ^ term;
    uint32_t s0      = c;
Intn=snprintf(out,cap,“AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPE
%u %u %u",s0, s1, s2);
```

0x4025b0 <secret_mix>:
mov %edi,%eax ; xor $0xe395,%edi ; shr $0x3,%eax ; xor $0x3d11,%eax ;
add %edi,%eax
用 C 语言表示就是 secret_mix(x) = ((x >> 3) ^ 0x3d11) + (x ^ 0xe395)

测试现有的 defuser 在 seed=42 的时候的第七行输出，把结果写入 secret.txt

```
sunny@sunny:~/ics-2026-bomblab$ ./defuser --seed 42 --secret > secret.txt
sunny@sunny:~/ics-2026-bomblab$ cat secret.txt
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN 3 2410714519 1604612190
```

secret.txt

```
1   2624475659
2   95 228 485 1021 2055 4135
3   4 381728391
4   1434892390
5   917923980 2642
6   1191021204
7   AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN 3 2410714519 1604612190
```

问题 2　输出　调试控制台　终端　端口 3　bash - ics-2026-bom

```
sunny@sunny:~/ics-2026-bomblab$ ./bomb --seed 42 < secret.txt
林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

测试别的 seed-54793462839

```
sunny@sunny:~/ics-2026-bomblab$ ./defuser --seed 54793462839 --secret
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN 1 1475479056 2640683838
```

./defuser --seed 54793462839> secret.txt 并输入刚得出的第七行

```
2059652060
64 172 379 804 1653 3342
5 2526718359
1539305555
3668109850 1622
2094168923
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN 1 1475479056 2640683838
```

```
问题 2  输出  调试控制台  终端  端口 3                bash - ics-2026-bomblab

sunny@sunny:~/ics-2026-bomblab$ ./bomb --seed 54793462839 < secret.txt

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

公开 seed 测试通过

```
sunny@sunny:~/ics-2026-bomblab$ python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

参考材料：

https://mp.weixin.qq.com/s/UBxW5G7VKvT85LKk1oJNJw

【读薄 CSAPP】贰 机器指令与程序优化 | 小土刀 4.0

https://mp.weixin.qq.com/s/OQbao-OtZwlNXuisJzqEGw

http://users.umiacs.umd.edu/~tudor/courses/ENEE757/Fall15/misc/gdb_tutorial.html

https://mp.weixinbridge.com/mp/wapredirect?url=https%3A%2F%2Fxunyidian.com%2Ft%2FGdbManual&action=appmsg_redirect&uin=MTAwMzA5NzMyOQ==&biz=Mzg4Mzg4NTUyNA==&mid=2247483866&idx=3&type=0&scene=0

https://mp.weixin.qq.com/s/fI6aZA7gaNyldL4h_WF_iA

AI 工具的使用用途：在自己过了汇编语言后，会询问 AI 看不懂的汇编命令，如 cmpl 的功能。此外，会让 AI 先解释 teaching 里的 C 语言代码。在调试有错误的时候，会让 AI 分析具体在哪一段出现错误。